Virtual Mouse — Human-friendly Documentation

A hand-gesture driven virtual mouse that turns your webcam into an air pointer. This project uses a camera + hand landmarks to move the OS cursor, click, drag, right-click, and scroll — all without touching the touchpad or mouse.

Quick overview

Virtual Mouse turns simple hand motions into natural desktop interactions. It's built for clarity and reliability rather than novelty: stable pointer smoothing, pinch-to-click with drag support, two-hand right-click + scroll, and a small HUD to confirm actions.

This README explains what the program does, why design choices were made, how to run it, and how to tune behaviour for different environments.

Repo

The code lives in the project repository: entity["software","Virtual_Mouse GitHub repo","github repo sasindusachintha/Virtual_Mouse"]. Check the repo for source, model asset, and sample recordings.

Dependencies

The project depends on a few external packages:

- entity["software","OpenCV","computer-vision library"] — camera capture, image display, and simple drawing overlays.
- entity["software","PyAutoGUI","python automation library"] — moving the OS cursor, clicks, drags and scrolling.
- entity["software","MediaPipe","google mediapipe handlandmarker"] — robust hand landmark detection (uses the `hand_landmarker.task` model file).

> Note: The repository expects a MediaPipe Task model file named `hand_landmarker.task` in the working directory (or adjust the code to point to the actual path). See the MediaPipe docs for how to export or download that model.

What it can do (features)

- Move the cursor using your right-most index fingertip.

- **Pinch** (thumb + index) to perform click, or hold to start a drag (with configurable delay).
- Two-hand gestures: left hand pinches to issue right-clicks (with cooldown), or left hand pinky-up to enable vertical scrolling.
- Cursor smoothing and a small deadzone to avoid jitter near frame edges.
- Simple HUD (on-screen text) to confirm actions like CLICK, DRAG, DROP, RIGHT CLICK, SCROLLING.

How it works — human explanation

1. The program captures a mirrored webcam frame.
2. A hand landmark detector finds up to two hands. The **rightmost** hand (by index-fingertip x coordinate) controls the mouse.
3. The right hand index finger position is mapped from camera coordinates into screen coordinates. A small margin is applied to avoid mapping the camera edges to extreme screen edges.
4. Recent pointer positions are averaged with weighted smoothing so the cursor moves naturally and resists camera jitter.
5. When the right hand thumb and index come close (pinch), the program either clicks (short pinch) or begins a drag (long pinch). When released, it either drops (ends drag) or issues the click.
6. If a second hand is present, left-hand gestures provide additional controls (right-click or scrolling).

This design keeps the pointer stable (smoothing + buffer) while still allowing fast motion when you move your hand quickly.

Tunable parameters (what to change and why)

All tuning values are defined near the top of the main script (with comments). Key parameters:

- `MARGIN` — keeps a boundary between camera edges and the screen edges. Increase if your hand is often clipped at the camera border.
- `PINCH_CLOSE` / `PINCH_OPEN` — thresholds (in pixels) to register a pinch and to release it. A small hysteresis gap avoids flicker when the fingers hover.
- `SMOOTH` — how eagerly the cursor follows the detected fingertip. Lower values produce smoother, slower-following cursor; higher values are snappier but can feel jittery.
- `DRAG_DELAY` — hold time (seconds) before a pinch becomes a drag. Increase to avoid accidental drags.
- `RIGHT_COOLDOWN` — minimum seconds between left-hand right-clicks to prevent accidental double right-clicks.
- `SCROLL_SENS` — multiplier for scroll speed when the left-hand pinky is raised.

- `HUD_DURATION` — how long brief HUD messages remain on the screen.

Tips: if your environment has lots of camera noise or low lighting, increase `SMOOTH` and `MARGIN`. If the cursor feels sluggish, decrease `SMOOTH`.

Running the project (recommended steps)

1. Create a Python virtual environment and activate it:

```
python -m venv venv
# Windows
venv\Scripts\activate
# macOS / Linux
source venv/bin/activate
```

2. Install dependencies:

```
pip install opencv-python pyautogui mediapipe
```

   If your OS blocks `pyautogui` actions, grant accessibility control (macOS) or run with appropriate permissions.

3. Place the MediaPipe `hand_landmarker.task` model in the project folder or update `model_asset_path` in the code to point to the model.

4. Run the script:

```
python main.py
```

5. The window is mirrored (flipped). Use your right hand to control the pointer. Press `ESC` to quit.

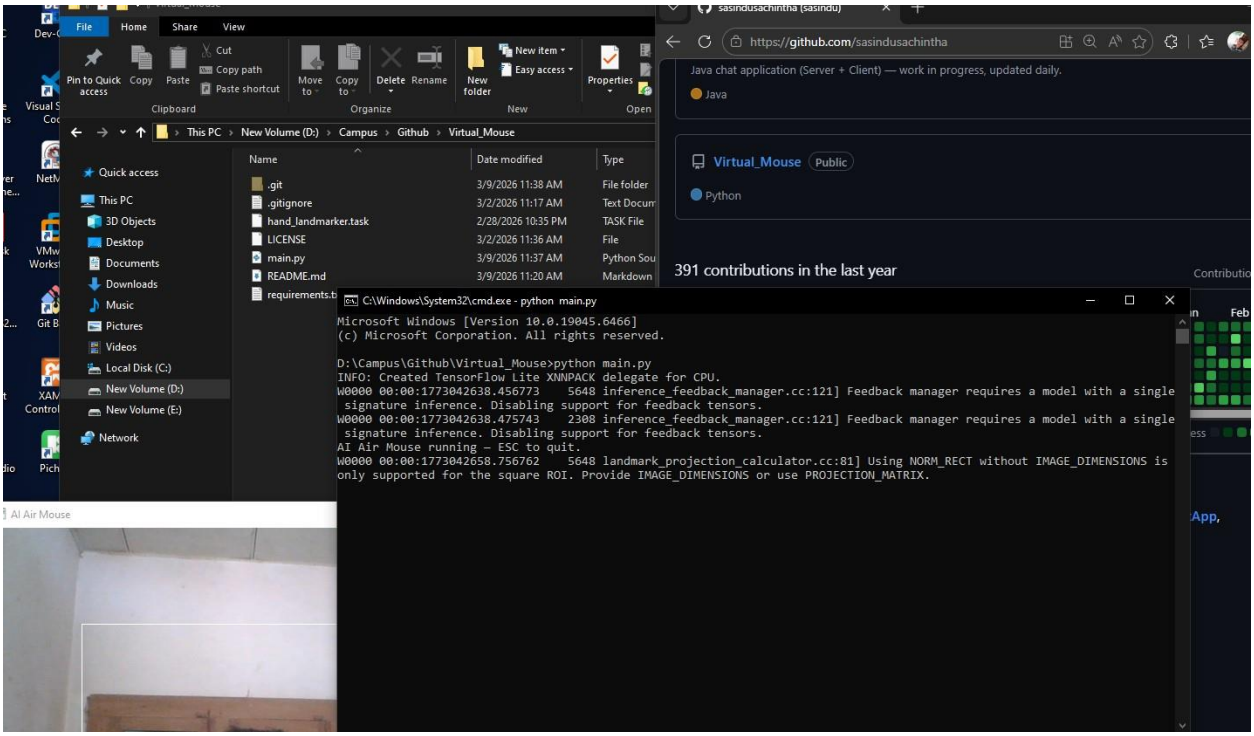

Figure 1: project execution ss

Important implementation notes (for contributors)

- **Rightmost-hand selection**: The code sorts detected hands by the x coordinate of the index fingertip and picks the rightmost one as the pointer hand. This avoids ambiguity when both hands are present.
- **Pinch logic with hysteresis**: The script uses separate `PINCH_CLOSE` and `PINCH_OPEN` values so a small finger tremor doesn't repeatedly open/close the pinch state.
- **Drag behaviour**: When the pinch becomes a drag, the code uses the `frozen_x` and `frozen_y` values (the cursor position where the pinch started) to click/drag consistently even while the hand moves.
- **Two stacks?** No — undo/redo is unrelated. (This note is for DSA students who may review the repo.)

Gesture mapping (summary)

- **Move**: Rightmost hand — index fingertip.
- **Left click**: Right-hand pinch (short press).
- **Drag**: Right-hand pinch held for `DRAG_DELAY` seconds (then move hand to drag until release).

- **Right click**: Left-hand pinch while two hands visible (cooldown enforced).
- **Scroll**: Left-hand pinky up with index finger vertical movement.

Safety, permission and UX

- On macOS, you must allow the terminal or Python interpreter Accessibility / Input Monitoring permission so `pyautogui` can control the cursor.
- `pyautogui.FAILSAFE` is set to `False` in the script because the natural mouse movements may otherwise trigger a failsafe; be careful: turning this off removes the emergency stop gesture. Run with caution.
- Test at low `SMOOTH` and increase gradually until it feels comfortable.

Troubleshooting

- **Cursor jumps or jitters**: raise `SMOOTH`, increase `pos_buffer` length, ensure good lighting and clean background.
- **Pinch not detected reliably**: adjust `PINCH_CLOSE` and `PINCH_OPEN`. Make sure the model file is correct and the camera resolution is adequate.
- **Right-clicks happen repeatedly**: increase `RIGHT_COOLDOWN`.
- **Scroll is too sensitive**: reduce `SCROLL_SENS`.

Extending the project

Ideas for future work:

- Use motion-based gestures (swipe left/right) for multimedia controls.
- Add calibration step to map a comfortable physical region to the full screen more precisely.
- Replace simple smoothing with an adaptive Kalman filter for even better stability.
- Add a minimal GUI to toggle features and tune parameters live.

Credits & License

- Hand detection powered by MediaPipe.
- Cursor control via PyAutoGUI.
- Camera & overlays via OpenCV.
- Licensed via under MIT License.